

Report
Of
ARTIFICIAL INTELLIGENCE INTERNSHIP PROJECTS

Submitted
To

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE



By

2410030259

AKARSH KUMAR

Batch: 3CSE2

2026-27

TABLE OF CONTENTS

Chapter / Section	Title	Page
	CANDIDATE'S DECLARATION	3
	ACKNOWLEDGEMENT	4
	INTERNSHIP COMPLETION CERTIFICATE	5
	LETTER OF RECOMMENDATION & OFFER LETTER	6
	LIST OF FIGURES	7
	LIST OF TABLES	7
4.1	Introduction	8
4.2	Organization Profile	8
4.3	Problem Statement	9
4.4	Project Objectives	9
4.5	Scope of the Project	9
4.6	Technologies and Tools Used	10
4.7	System Architecture	10
4.8	Methodology	10, 11
4.9	Expected Outcomes	11
4.10	AI FAQ Chatbot	12
4.11	AI Object Detection and Tracking	16
4.12	AI Language Translation Tool	20
4.13	Consolidated Learning Outcomes	23
4.14	Overall Conclusion	23
4.15	Personal Internship Reflection	24
4.16	Practical Learning and Challenges	25
5	References	26

CANDIDATE’S DECLARATION

I, Akarsh Kumar, hereby declare that the internship report titled “Artificial Intelligence Internship Projects” has been prepared by me to fulfil the academic requirements of the Bachelor of Technology in Computer Science and Engineering. The report describes the work completed during the CodeAlpha Artificial Intelligence Internship Program and presents the technical concepts, implementation approach, project structure and learning outcomes of the applications covered.

The report has been prepared using the project documentation and implementation information available for the internship tasks. Technical references and software documentation used for understanding the tools have been acknowledged in the bibliography. I accept responsibility for the originality and academic integrity of the submitted report.

Signature: Akarsh Kumar

Student Name: Akarsh Kumar

Roll No.: 2410030259

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to CodeAlpha for providing me with the opportunity to participate in the Artificial Intelligence Internship Program. The internship gave me practical exposure to different areas of AI and helped me understand how concepts learned during my B.Tech course can be applied while developing actual software applications.

I am thankful to the School of Computer Science and Engineering, IILM University, Greater Noida, for providing the academic environment required to complete this work. I also appreciate the guidance and encouragement received from my teachers during my academic journey.

I am equally grateful to my parents, family members and friends for their continuous support. Their encouragement helped me remain motivated while learning new frameworks, solving implementation problems and preparing this report.

Signature: Akarsh Kumar

Student Name: Akarsh Kumar

Roll No.: 2410030259

INTERNSHIP COMPLETION CERTIFICATE

 YOUR DREAM OUR PASSION	CERTIFICATE OF COMPLETION	 Student ID: CA/DF1/113561
This Certificate Is Proudly Presented To		
<i>Akarsh Kumar</i>		
Was an active Participant at CodeAlpha Virtual Internship Program in Artificial Intelligence Internship with dedication and hard work. We really appreciate your efforts taken and wish you all the best for the further. 10th June 2026 to 10th July 2026		
 CEO & Founder	11th July 2026 Date of Issue	 MINISTRY OF MICRO, SMALL & MEDIUM ENTERPRISES

LETTER OF RECOMMENDATION & OFFER LETTER



CODEALPHA

YOUR DREAM OUR PASSION

Letter Of Recommendation

TO WHOM IT MAY CONCERN

This is to certify that Akarsh Kumar has successfully completed the Virtual Internship Program at CodeAlpha for a duration of one month from 10th June 2026 to 10th July 2026 as a Artificial Intelligence Internship.

He/She exhibited performance in this role and made a valuable contribution to our organization during the internship period. He/She has excellent analytical skills and He/She quickly acquired new skills and adeptly adapted to emerging technologies, demonstrating a high level of productivity.

He/She consistently displayed a willingness to provide assistance and fostered strong collaboration with fellow team members. His/Her dedication and skills make him/her a valuable asset to any prospective employer, and CodeAlpha wholeheartedly recommend him/her for any future endeavor.

REGARDS,

CA/DF1/113561

STUDENT ID


FOUNDER & CEO



11th July 2026

DATE

Website:

www.codealpha.tech

Email ID:

services@codealpha.tech
support@codealpha.tech

LinkedIn:

www.linkedin.com/company/codealpha

Mobile No:

+91 9336576683



INTERNSHIP OFFER LETTER

2nd June 2026

STUDENT ID: CA/DF1/113561

Name: Akarsh Kumar

Welcome to CodeAlpha

Congratulations! We are delighted to make you an offer as **Artificial Intelligence Internship** offer letter is to confirm your selection **Artificial Intelligence Internship** at **CodeAlpha** and welcome you for the same effective from 10th June 2026 to 10th July 2026. Hope you will be doing well.

This Internship is observed by CodeAlpha as being a learning opportunity for you.

Your internship will embrace orientation and give emphasis on learning new skills with a deeper understanding of concepts through hands-on application of the knowledge you gained as an intern. You will acknowledge your obligation to perform all work allocated to you to the best of your ability within lawful and reasonable direction given to you.

We look forward to a worthwhile and fruitful association which will make you equipped for future projects.

Wishing you the most enjoyable and truly meaningful internship program experience.

Sincerely,


Swati Srivastava
Founder & CEO



Phone
+91 9336576683

Email
services@codealpha.tech

Address
Lucknow U.P.

LIST OF FIGURES

Figure	Title
Figure 4.1	Overall internship project architecture
Figure 4.2	AI FAQ Chatbot architecture
Figure 4.3	AI FAQ Chatbot workflow
Figure 4.4	AI Object Detection and Tracking architecture
Figure 4.5	AI Object Detection and Tracking workflow
Figure 4.6	AI Language Translation Tool architecture
Figure 4.7	AI Language Translation Tool workflow

LIST OF TABLES

Table	Title
Table 4.1	Technologies and tools used across the internship projects
Table 4.2	AI FAQ Chatbot technology mapping
Table 4.3	AI Object Detection and Tracking technology mapping
Table 4.4	AI Language Translation Tool technology mapping
Table 4.5	Consolidated learning outcomes

CHAPTER 4

PROJECT DESCRIPTION

4.1 Introduction

The internship consisted of three Artificial Intelligence projects developed under the CodeAlpha Artificial Intelligence Internship Program. The projects were deliberately different from one another so that the internship provided exposure to multiple areas of AI. The first project focused on Natural Language Processing and information retrieval, the second on Computer Vision and real-time object tracking, and the third on multilingual text translation.

The work was useful because it required more than writing isolated code. Each project involved understanding a requirement, selecting appropriate technologies, arranging the files and dependencies, creating a user-facing application and documenting how the system works. This helped connect academic concepts with practical software development.

The three applications also show different approaches to AI. The FAQ chatbot relies on numerical representation and similarity rather than a large trained language model. The object detection system uses a pre-trained YOLOv8 model for visual recognition. The translation application integrates a translation engine through a simple interactive interface. Together, they provide a broad introduction to applied AI development.

4.2 Organization Profile

The organization identified in the supplied project documentation is CodeAlpha, with the internship domain specified as Artificial Intelligence. The supplied README files identify the three tasks as an AI FAQ Chatbot, AI Object Detection and Tracking, and an AI Language Translation Tool.

- The internship followed a practical, task-oriented approach.
- The projects required independent implementation and documentation.
- The work involved commonly used Python AI libraries and frameworks.
- The tasks provided experience in converting a technical requirement into a working application.

4.3 Problem Statement

The projects address three practical problems. First, users need an easier way to obtain answers from an FAQ dataset when their wording does not exactly match a stored question. Second, users need automated visual information from a live camera stream rather than relying only on raw video. Third, users need a simple way to translate text between languages without manually handling the underlying translation process.

- FAQ problem: Traditional FAQ pages can require manual searching and exact or near-exact wording.
- Vision problem: A raw video stream does not identify, classify or track objects for the viewer.
- Translation problem: Language differences can make digital information difficult to understand.

4.4 Project Objectives

- Build a practical AI FAQ chatbot using NLP and similarity matching.
- Understand TF-IDF and cosine similarity through implementation.
- Develop a real-time object detection and tracking application.
- Learn how YOLOv8 and OpenCV work together in a computer-vision pipeline.
- Build an interactive translation application.
- Understand automatic source-language detection and target-language selection.
- Improve Python programming, debugging, testing and documentation skills.

4.5 Scope of the Project

- The report covers the three internship projects described in the supplied README material.
- It explains the objectives, features, technologies, architecture and workflow of each project.
- It includes the project structures and the learning outcomes described in the documentation.
- It discusses future improvements suggested for each project.
- It avoids claiming numerical accuracy, FPS or other measurements that were not provided.

4.6 Technologies and Tools Used

Technology	Purpose
Python	Core programming language across the applications.
Streamlit	Interactive web interface for the FAQ chatbot and translation tool.
Pandas	FAQ dataset processing.
Scikit-learn	TF-IDF vectorization and cosine similarity.
NLTK	Text-processing support.
OpenCV	Webcam and video processing.
YOLOv8 / Ultralytics	Object detection and tracking.
NumPy	Numerical operations.
deep-translator	Translation functionality.
Pyperclip	Clipboard support.
CSV	FAQ knowledge-base format.

Table 4.1. Technologies and tools used across the internship projects.

4.7 System Architecture

At a conceptual level, the three applications follow an input-processing-output pattern. The input may be text or a live video frame. The processing stage prepares the input and passes it to the relevant AI component. The final stage presents the result in a form that can be understood by the user.

- Input layer: Question, text or camera frame.
- Processing layer: Preprocessing, vectorization, frame handling or language detection.
- AI layer: Similarity matching, YOLO detection/tracking or translation.
- Application layer: Python code connecting the AI operation with the interface.
- Output layer: Answer, annotated video or translated text.

4.8 Methodology

- Requirement analysis: Understand what the application should do and what the user expects.
- Technology selection: Select an appropriate library, model or service for the requirement.
- Implementation: Build the application using modular Python code and the required dependencies.

- Testing: Use representative inputs and observe whether the expected output is produced.
- Documentation: Record the project structure, workflow, installation process and future improvements.

4.9 Expected Outcomes

- A usable FAQ chatbot capable of finding relevant stored answers.
- A real-time object detection and tracking application.
- A multilingual translation application.
- Practical understanding of NLP, Computer Vision and language-processing workflows.
- Better confidence in Python-based AI application development.

4.10 AI FAQ Chatbot

4.10.1 Project Overview

The AI FAQ Chatbot is an AI-powered Frequently Asked Questions application developed using Python, Natural Language Processing, TF-IDF vectorization, cosine similarity and Streamlit. Its purpose is to match a user's natural-language question with the most relevant question stored in an FAQ knowledge base and then return the associated answer. The project is a useful example of a lightweight information-retrieval system. Instead of requiring an exact text match, the application represents questions numerically and compares them. The README also identifies a Streamlit live deployment and a GitHub repository for the project.

4.10.2 Problem Statement

A traditional FAQ page can make users search through many questions manually. Another difficulty is that a user may ask a question using wording that differs from the stored FAQ. The project attempts to make this interaction more natural by comparing the user's wording with the available FAQ questions and selecting the closest match.

4.10.3 Objectives

- Create an interactive FAQ interface.
- Use NLP-based text representation.
- Convert questions into TF-IDF vectors.
- Calculate similarity between the user question and stored questions.
- Return the answer associated with the best match.
- Keep the FAQ knowledge base easy to customize.

4.10.4 Features

- Intelligent FAQ matching
- Natural Language Processing
- TF-IDF vectorization
- Cosine-similarity response generation
- Interactive chat interface
- Fast response time
- User-friendly design

- Customizable FAQ dataset

4.10.5 Technology Mapping

Technology	Purpose
Python	Core implementation.
Streamlit	Frontend user interface.
Pandas	Dataset processing.
Scikit-learn	TF-IDF and cosine similarity.
NLTK	Text-processing support.
CSV	FAQ knowledge base.

Table 4.2. AI FAQ Chatbot technology mapping.

4.10.6 Project Structure

```

CodeAlpha_FAQChatbot/
|
├── assets/
|   ├── home.png
|   └── chat.png
|
├── chatbot/
|   ├── __init__.py
|   └── faq_engine.py
|
├── data/
|   └── faq.csv
|
├── app.py
├── requirements.txt
└── README.md

```

4.10.7 Methodology and Workflow

- Step 1: The user enters a question such as “What are your working hours?”
- Step 2: The input is prepared for comparison with the stored FAQ questions.
- Step 3: TF-IDF converts the questions into numerical vectors.
- Step 4: Cosine similarity compares the user vector with the FAQ vectors.
- Step 5: The most relevant FAQ is selected.
- Step 6: The associated answer is displayed in the chat interface.

4.10.8 Future Enhancements

- Voice-based chatbot
- Multi-language support
- Text-to-speech responses
- Large Language Model integration
- Chat history
- Dark mode
- Database integration
- Export chat conversations

4.10.9 Learning Outcomes

- Natural Language Processing
- Cosine similarity
- Information retrieval
- Streamlit development
- Dataset management
- AI chatbot design
- TF-IDF vectorization

4.10.10 NLP Concepts Used

- **TF-IDF:** Term Frequency–Inverse Document Frequency converts text into numerical values that represent the relative importance of words. It is useful when a collection of questions needs to be compared because common terms are less useful for distinguishing between questions.
- **Cosine Similarity:** Once the questions have been represented as vectors, cosine similarity is used to compare their orientation. This makes it suitable

for finding a question whose wording is close to the user's wording.

- **Intent Matching:** The selected FAQ is treated as the closest representation of what the user is asking. The answer linked to that FAQ is then returned.

4.10.11 Example Questions

- What are your working hours?
- Where are you located?
- How can I contact support?
- Do you provide internships?
- What services do you offer?
- Can I work remotely?
- Do you provide certificates?

4.11 AI Object Detection and Tracking

4.11.1 Project Overview

The AI Object Detection and Tracking system is a real-time Computer Vision application developed using Python, OpenCV and YOLOv8. It uses a webcam feed to detect, classify and track multiple objects while assigning unique tracking IDs and displaying object counts and confidence scores. The project uses YOLOv8n (Nano), which the supplied README describes as a lightweight model intended for real-time use. The project demonstrates how a pre-trained detection model can be integrated into a live video-processing application.

4.11.2 Problem Statement

A webcam provides a sequence of images, but raw video does not automatically identify the objects visible in those images. A useful monitoring application needs to locate objects, classify them and maintain their identity while they move between frames. This project addresses that requirement through detection and tracking.

4.11.3 Objectives

- Process live webcam frames.
- Detect multiple objects using YOLOv8.
- Classify detected objects.
- Display bounding boxes and confidence scores.
- Assign unique tracking IDs.
- Maintain identity across frames.
- Display object counts.

4.11.4 Features

- Real-time object detection
- Multi-object tracking
- Unique tracking IDs
- Live webcam processing
- Object counting
- Bounding-box visualization
- Confidence score display
- Lightweight YOLOv8 model
- Screenshot capture support

4.11.5 Technology Mapping

Technology	Purpose
Python	Core implementation.
OpenCV	Webcam and video processing.
YOLOv8	Object-detection model.
Ultralytics	YOLO framework and tracking.
NumPy	Numerical operations.
Pandas	Listed dependency in the README.

Table 4.3. AI Object Detection and Tracking technology mapping.

4.11.6 Project Structure

```
CodeAlpha_ObjectDetectionTracking/  
|  
├─ app.py  
├─ requirements.txt  
├─ README.md  
|  
├─ assets/  
|   ├─ detection.png  
|   └─ tracking.png  
|  
└─ models/  
    └─ yolov8n.pt
```

4.11.7 Methodology and Workflow

- Step 1: The webcam captures live frames.
- Step 2: OpenCV supplies the frames to the processing pipeline.
- Step 3: YOLOv8 detects and classifies objects.
- Step 4: The tracker assigns or maintains a unique ID.
- Step 5: Bounding boxes, labels and confidence values are drawn.
- Step 6: The processed frame is displayed in real time.

4.11.8 Future Enhancements

- Video-file processing
- Detection history logging
- Object-counting dashboard
- Detection statistics

- Email alert system
- Face-recognition integration
- Streamlit web interface
- Cloud deployment

4.11.9 Learning Outcomes

- Computer Vision
- Deep-learning model integration
- Object detection
- Object tracking
- OpenCV development
- Real-time video processing
- Ultralytics YOLO framework

4.11.10 Supported Object Categories

The README states that the YOLOv8 model can detect more than 80 object categories. Examples specifically listed include:

- Person
- Cell Phone
- Laptop
- Chair
- Bottle
- Keyboard
- Mouse
- TV
- Car
- Bicycle
- Dog
- Cat

4.11.11 Example Tracking Output

```
id:403 person 0.83  
id:160 cell phone 0.38  
id:66 chair 0.70
```

4.11.12 Performance Discussion

- The selected model is YOLOv8n (Nano).
- The project documentation describes it as lightweight.
- The README emphasizes fast inference and real-time use.
- No numerical FPS, accuracy or latency values were supplied, so this report does not invent them.

4.12 AI Language Translation Tool

4.12.1 Project Overview

The AI Language Translation Tool is a modern Python and Streamlit application developed using deep-translator. It automatically detects the source language and translates the input text into a language selected by the user. The project focuses on making translation accessible through a simple interface rather than requiring users to interact with a translation service directly.

4.12.2 Problem Statement

Language differences can create difficulties when users need to read or communicate information written in another language. The application provides a simple workflow in which the user enters text, chooses the target language and receives a translated result. Automatic source-language detection reduces the amount of information the user needs to provide manually.

4.12.3 Objectives

- Automatically detect the source language.
- Translate text into a selected language.
- Support multiple languages.
- Provide an interactive interface.
- Allow translated text to be copied.
- Keep the application lightweight and easy to use.

4.12.4 Features

- Automatic language detection
- Real-time translation
- Multiple language support
- Copy translation to clipboard
- Clean interactive UI
- Fast and lightweight operation

4.12.5 Technology Mapping

Technology	Purpose
Python	Core programming.
Streamlit	Frontend interface.
deep-translator	Translation engine.
Pyperclip	Clipboard functionality.

Table 4.4. AI Language Translation Tool technology mapping.

4.12.6 Project Structure

```
LanguageTranslationTool/  
|  
├─ assets/  
|   ├── home.png  
|   └─ translation.png  
|  
├─ utils/  
|   ├── __init__.py  
|   ├── languages.py  
|   └─ translator.py  
|  
├─ app.py  
├─ README.md  
└─ requirements.txt
```

4.12.7 Methodology and Workflow

- Step 1: The user enters text in the application.
- Step 2: The user selects the target language.
- Step 3: The application detects the source language automatically.
- Step 4: The translation engine processes the text.
- Step 5: The translated output is displayed.
- Step 6: The user can copy the translated result.

4.12.8 Future Enhancements

- Speech-to-text translation
- Text-to-speech output
- Dark mode support
- Translation history
- Export translation as PDF
- Additional language support

4.12.9 Learning Outcomes

- Translation tools and APIs
- Natural Language Processing concepts
- Python application development
- Streamlit framework
- API integration
- User-interface design

4.12.10 Supported Languages

- English
- Hindi
- French
- German
- Spanish
- Italian
- Japanese
- Chinese
- Russian
- Arabic
- Portuguese
- Korean

4.12.11 Application Workflow in Practice

A typical interaction starts when the user enters a sentence in the input area. The target language is then selected from the available options. The application determines the source language and sends the text through the translation engine. The resulting translation is presented in the interface, where it can be copied for use elsewhere.

4.13 Consolidated Learning Outcomes

Area	Learning demonstrated
Natural Language Processing	Text representation, similarity matching and language-processing workflows.
Computer Vision	Real-time detection, classification and tracking.
AI model integration	Using established models and libraries inside Python applications.
Web application development	Creating interactive interfaces with Streamlit.
Data management	Using structured FAQ data and project resources.
Software organization	Separating application code, utilities, datasets, assets and dependencies.
Documentation	Explaining installation, workflow, structure, features and future improvements.

Table 4.5. Consolidated learning outcomes.

4.14 Overall Conclusion

The three projects provided complementary practical experience in Artificial Intelligence. The FAQ chatbot introduced Natural Language Processing and similarity-based information retrieval. The object detection project provided experience with Computer Vision and real-time model inference. The translation tool demonstrated how language-processing services can be connected to an interactive application.

A major learning from the internship was that developing an AI application involves several connected stages. Selecting an algorithm or library is only one part of the process. The input must be handled correctly, the processing workflow must be organized, the output must be presented clearly and the application must be documented so that another person can understand and run it.

The internship also improved my confidence in working independently with Python libraries and frameworks. The projects created a practical foundation for future work involving larger datasets, more advanced models, improved user interfaces, persistent storage, analytics and cloud deployment.

4.15 PERSONAL INTERNSHIP REFLECTION

One of the most useful parts of this internship was getting an opportunity to move beyond classroom examples and work on applications that have a clear purpose. Before working on these projects, concepts such as TF-IDF, cosine similarity, object detection and translation libraries were mainly technical topics that I understood from study material. Implementing them made the concepts easier to understand because I could see what role each component played in a complete application.

The three projects also helped me realise that an AI project is not only about the model or algorithm. A working application needs a suitable input method, clean processing logic, a clear output and a user interface that makes the result understandable. While working through the projects, I had to think about how the individual libraries would fit together instead of treating them as separate pieces of code.

In the FAQ chatbot, I particularly understood why similarity-based matching can be useful. A user may ask a question in their own words, while the stored FAQ may use different wording. Representing the questions numerically and comparing them gave me a practical understanding of information retrieval. It also showed me that the quality of the underlying FAQ data is important because the system can only return useful answers when the knowledge base is properly prepared.

The object detection and tracking project gave me a different kind of experience because the input was continuously changing video rather than a fixed text query. I learned how a webcam frame becomes the starting point of a computer-vision pipeline and how detection, classification and tracking work together. Seeing object IDs remain associated with objects across frames made the idea of tracking much more concrete than simply reading about it.

The translation tool helped me understand another side of AI application development: integrating an existing language-processing service into a user-friendly program. The implementation was relatively lightweight, but it still involved input handling, language selection, source-language detection, translation and presentation of the result. This project reinforced the importance of keeping the interface simple when the underlying technology is already complex.

4.16 PRACTICAL LEARNING AND CHALLENGES

During the internship, I also learned that small implementation decisions can affect the overall usability of an application. Organising files into separate modules, keeping datasets and assets in appropriate folders, installing the correct dependencies and documenting the execution steps may look like minor tasks, but they become important when a project has to be shared or demonstrated to another person.

Some of the practical areas that required attention were:

- Understanding unfamiliar libraries: Each project used a different set of tools, so I had to understand the purpose of the library before using it effectively.
- Connecting components: The AI technique had to be connected with Python application logic and, where applicable, a Streamlit interface or webcam pipeline.
- Handling user input: The applications needed to respond to natural-language questions, typed translation requests or continuously changing camera frames.
- Keeping the project maintainable: Separating application files, utility modules, datasets, assets and model files made the projects easier to understand.
- Testing the complete flow: It was important to think about the complete journey from user input to final output rather than checking individual functions in isolation.

The internship also improved my confidence in debugging and learning independently. When a tool or concept was unfamiliar, I became more comfortable reading documentation, understanding examples and then adapting the idea to the project requirement. This is one of the skills I consider most valuable from the internship because software development regularly requires learning technologies that were not covered in detail in class.

Overall, the internship gave me a more realistic understanding of applied Artificial Intelligence. I now have a clearer view of how NLP, Computer Vision and language-processing technologies can be turned into small but complete applications. The experience has also encouraged me to explore more advanced AI systems in the future, particularly systems involving larger datasets, stronger models, persistent storage, analytics and cloud deployment.

This report therefore represents not only the final project outputs, but also the practical learning process behind them. The intention has been to document the work in a clear and honest manner while reflecting the skills and understanding gained during the internship.

CHAPTER 5 REFERENCES

- [1] Python Software Foundation, “Python Documentation,” <https://docs.python.org/3/>
- [2] Streamlit, “Streamlit Documentation,” <https://docs.streamlit.io/>
- [3] Scikit-learn Developers, “Metrics and Similarity Documentation,” <https://scikit-learn.org/stable/modules/metrics.html>
- [4] Ultralytics, “Ultralytics YOLO Documentation,” <https://docs.ultralytics.com/>
- [5] Ultralytics, “Multi-Object Tracking Documentation,” <https://docs.ultralytics.com/modes/track>
- [6] OpenCV, “OpenCV Documentation,” <https://docs.opencv.org/>
- [7] deep-translator, “deep-translator Documentation,” <https://deep-translator.readthedocs.io/>
- [8] Akarsh Kumar, “CodeAlpha_FAQChatbot,” GitHub repository, https://github.com/AkarshKumar1/CodeAlpha_FAQChatbot
- [9] Akarsh Kumar, “CodeAlpha_ObjectDetectionTracking,” GitHub repository, https://github.com/AkarshKumar1/CodeAlpha_ObjectDetectionTracking
- [10] Akarsh Kumar, “CodeAlpha_LanguageTranslationTool,” GitHub repository, https://github.com/AkarshKumar1/CodeAlpha_LanguageTranslationTool
- [11] CodeAlpha, Artificial Intelligence Internship Project Documentation, README material supplied for this report.